

Ejercicios_propuestos_resuelto_junio_2026

July 5, 2026

1 Práctica 4: Ejercicios propuestos

2 Inteligencia Artificial

3 Grado en Ingeniería Informática - Ingeniería del Software

4 Universidad de Sevilla

4.0.1 Ejercicio 1

La biblioteca `Unified Planning` permite leer un dominio de planificación automática a partir de un fichero PDDL y crear posteriormente instancias de ese dominio mediante la interfaz proporcionada por la biblioteca.

En este ejercicio se pretende seguir esa metodología para crear instancias del mundo de los bloques que contengan los bloques B_0 a B_{N-1} , apilados inicialmente en ese orden, y en la que el objetivo sea que estén apilados al contrario, con el bloque B_{N-1} sobre la mesa, el bloque B_{N-2} sobre el B_{N-1} , el B_{N-3} sobre el B_{N-2} , etc.

Se pide realizar lo siguiente:

1. Leer el dominio del mundo de los bloques a partir del fichero `dominio_mundo_bloques.pddl`.

```
[1]: from unified_planning.io import PDDLReader
```

```
[4]: lector_PDDL = PDDLReader()
      dominio_mundo_bloques = lector_PDDL.parse_problem("dominio_mundo_bloques.pddl")
```

```
[6]: print(dominio_mundo_bloques)
```

```
problem name = dominio_mundo_bloques
```

```
types = [object]
```

```
fluents = [
    bool sobre_la_mesa[b=object]
    bool sobre[b1=object, b2=object]
    bool agarrado[b=object]
    bool brazo_libre
    bool despejado[b=object]
```

```

]

actions = [
  action agarrar(object b) {
    preconditions = [
      (sobre_la_mesa(b) and despejado(b) and brazo_libre)
    ]
    effects = [
      sobre_la_mesa(b) := false
      despejado(b) := false
      brazo_libre := false
      agarrado(b) := true
    ]
  }
  action bajar(object b) {
    preconditions = [
      agarrado(b)
    ]
    effects = [
      agarrado(b) := false
      sobre_la_mesa(b) := true
      despejado(b) := true
      brazo_libre := true
    ]
  }
  action desapilar(object b1, object b2) {
    preconditions = [
      (sobre(b1, b2) and despejado(b1) and brazo_libre)
    ]
    effects = [
      sobre(b1, b2) := false
      despejado(b1) := false
      brazo_libre := false
      agarrado(b1) := true
      despejado(b2) := true
    ]
  }
  action apilar(object b1, object b2) {
    preconditions = [
      (agarrado(b1) and despejado(b2))
    ]
    effects = [
      agarrado(b1) := false
      despejado(b2) := false
      sobre(b1, b2) := true
      despejado(b1) := true
      brazo_libre := true
    ]
  }
]

```

```

    }
]

objects = [
    object: []
]

initial fluents default = [
    bool sobre_la_mesa[b=object] := false
    bool sobre[b1=object, b2=object] := false
    bool agarrado[b=object] := false
    bool brazo_libre := false
    bool despejado[b=object] := false
]

initial values = [
]

goals = [
]

```

2. Completar la definición de la función `crea_instancia_mundo_bloques`, sustituyendo los ... por código adecuado para que, dado el número N de bloques, la función proporcione el problema del mundo de los bloques descrito anteriormente.

```
[7]: from unified_planning.shortcuts import *
```

```
[8]: def crea_instancia_mundo_bloques(N):
    instancia = dominio_mundo_bloques.clone() # Trabajamos con una copia del
    dominio
    # Se añaden los objetos de la instancia
    block_type = instancia.user_type('object')

    ## Aquí empieza mi código que sustituye a la línea comentada de abajo
    # ...

    # Solución: Creamos los objetos
    instancia.add_objects([ Object(f"B{i}",block_type) for i in range(N) ])

    ## Aquí termina mi código

    # Se establece el estado inicial de la instancia
    sobre_la_mesa = dominio_mundo_bloques.fluent('sobre_la_mesa')
    sobre = dominio_mundo_bloques.fluent('sobre')
    agarrado = dominio_mundo_bloques.fluent('agarrado')
    brazo_libre = dominio_mundo_bloques.fluent('brazo_libre')

```

```

despejado = dominio_mundo_bloques.fluent('despejado')

## Aquí empieza mi código que sustituye a la línea comentada de abajo
# ...

lista_bloques = list(instancia.all_objects)

# 1. Brazo libre al inicio
instancia.set_initial_value(brazo_libre(), True)

#2. B0 sobre la mesa, B1 sobre B0, B2 sobre B1,... BN sobre BN-1
bloque_anterior = None
for bloque in lista_bloques:
    if bloque_anterior == None:
        instancia.set_initial_value(sobre_la_mesa(bloque), True)
    else:
        instancia.set_initial_value(sobre(bloque, bloque_anterior), True)

    bloque_anterior = bloque

#3. último bloque (BN) despejado
instancia.set_initial_value(despejado(lista_bloques[-1]), True)

# Aquí termina mi código

# Se establece el objetivo de la instancia

## Aquí empieza mi código que sustituye a la línea comentada de abajo
#...
# BN sobre la mesa, BN-1 sobre BN-2,... , B1 sobre B0
bloque_anterior = None
for bloque in reversed(lista_bloques):
    if bloque_anterior == None:
        instancia.add_goal(sobre_la_mesa(bloque))
    else:
        instancia.add_goal(sobre(bloque, bloque_anterior))

    bloque_anterior = bloque
# Aquí termina mi código

return instancia

instancia = crea_instancia_mundo_bloques(7)

print("--- ESTADO INICIAL ---")
for fluido, valor in instancia.initial_values.items():

```

```

# En unified_planning, los valores a veces son objetos FNode,
# así que podemos verificar si equivalen a True
if str(valor) == "true" or valor is True:
    print(f" {fluido}")
elif str(valor) != "false" and valor is not False:
    # Por si tienes fluentes numéricos (ej. altura, batería)
    print(f" {fluido} = {valor}")
print("-----")

print("--- ESTADO OBJETIVO ---")
for meta in instancia.goals:
    print(f" {meta}")
print("-----")

```

--- ESTADO INICIAL ---

```

brazo_libre
sobre_la_mesa(B0)
sobre(B1, B0)
sobre(B2, B1)
sobre(B3, B2)
sobre(B4, B3)
sobre(B5, B4)
sobre(B6, B5)
despejado(B6)

```

--- ESTADO OBJETIVO ---

```

sobre_la_mesa(B6)
sobre(B5, B6)
sobre(B4, B5)
sobre(B3, B4)
sobre(B2, B3)
sobre(B1, B2)
sobre(B0, B1)

```

3. Usar el planificador Fast Downward para tratar de resolver la instancia con el mayor número posible de bloques.

```

[9]: import time
from unified_planning.shortcuts import OneshotPlanner, get_environment

get_environment().credits_stream = None

# Tamaños de torre que vamos a probar
tamanos_a_probar = [5, 10, 20, 30, 50, 100, 200, 500]

print(" Iniciando Prueba de Estrés - Mundo de Bloques")

```

```

estados_resueltos = {"SOLVED_SATISFICING", "SOLVED_OPTIMALLY"}

for N in tamanos_a_probar:
    print(f"\n--- Probando con N = {N} bloques ---")
    instancia = crea_instancia_mundo_bloques(N)

    # Abrimos el planificador
    with OneshotPlanner(name="fast-downward") as planner:
        inicio = time.time()

        # Máximo de 30 segundos por problema
        resultado = planner.solve(instancia, timeout=30)

        tiempo_total = time.time() - inicio
        estado = resultado.status
        estado_nombre = getattr(estado, "name", str(estado))

        # Verificamos si logró encontrar una solución válida
        if estado_nombre in estados_resueltos:
            longitud_plan = len(resultado.plan.actions)
            print(f" ¡Resuelto! Tiempo: {tiempo_total:.2f} segundos.")
            print(f" Acciones necesarias: {longitud_plan}")
        else:
            print(f" Falló o excedió el tiempo límite (Estado:␣
↪{estado_nombre}).")
            print(" Límite de la máquina alcanzado.")
            break # Rompemos el bucle, ya no intentamos con N más grandes

```

Iniciando Prueba de Estrés - Mundo de Bloques

```

--- Probando con N = 5 bloques ---
¡Resuelto! Tiempo: 0.69 segundos.
Acciones necesarias: 10

--- Probando con N = 10 bloques ---
¡Resuelto! Tiempo: 0.56 segundos.
Acciones necesarias: 20

--- Probando con N = 20 bloques ---
¡Resuelto! Tiempo: 0.94 segundos.
Acciones necesarias: 40

--- Probando con N = 30 bloques ---
¡Resuelto! Tiempo: 1.26 segundos.
Acciones necesarias: 60

--- Probando con N = 50 bloques ---
¡Resuelto! Tiempo: 3.47 segundos.

```

```

Acciones necesarias: 100

--- Probando con N = 100 bloques ---
¡Resuelto! Tiempo: 16.20 segundos.
Acciones necesarias: 200

--- Probando con N = 200 bloques ---
Falló o excedió el tiempo límite (Estado: TIMEOUT).
Límite de la máquina alcanzado.

```

4.0.2 Ejercicio 2

En el marco de la *Conferencia Internacional sobre Planificación Automática y Planificación Temporal* ([International Conference on Automated Planning and Scheduling, ICAPS](https://www.icaps-conference.org/competitions/)) se celebra, con periodicidad aproximadamente trienal, la *Competición Internacional de Planificación* (<https://www.icaps-conference.org/competitions/>).

Esta competición tiene diferentes objetivos: realizar una comparación empírica del estado del arte de los sistemas de planificación; destacar desafíos para la comunidad de Planificación Automática; proponer nuevas direcciones para la investigación y nuevos vínculos con otros campos de la Inteligencia Artificial; y proporcionar nuevos conjuntos de datos que puedan ser utilizados por la comunidad científica como puntos de referencia.

Uno de los dominios utilizados en la competición del año 2002 combinaba el mundo de los bloques con la distribución logística de cajas. En este dominio hay una serie de camiones (que asumimos con capacidad infinita) que transportan cajas entre distintos lugares (que asumimos que están todos conectados entre sí); en esos lugares hay unos palés, sobre los que las cajas se colocan apiladas; los apilamientos se realizan con *polipastos* (hay al menos uno en cada lugar).

En este ejercicio se pide completar la especificación del dominio que se proporciona a continuación, sustituyendo en las acciones los ... por hechos adecuados, y tratar de resolver la mayor cantidad posible de las instancias de problemas proporcionadas en la carpeta Depot.

```

[10]: from unified_planning.shortcuts import *

[18]: dominio_depot = Problem('Depot')

[19]: # Jerarquía de tipos de objetos

Place = UserType('Place') # Lugar
Locatable = UserType('Locatable') # Ubicable
Depot = UserType('Depot', Place) # Almacén (Tipo de lugar)
Distributor = UserType('Distributor', Place) # Distribuidor (Tipo de lugar)
Truck = UserType('Truck', Locatable) # Camión (Tipo de ubicable)
Hoist = UserType('Hoist', Locatable) # Polipasto (Tipo de ubicable)
Surface = UserType('Surface', Locatable) # Superficie (Tipo de ubicable)
Pallet = UserType('Pallet', Surface) # Palé (Tipo de superficie)
Crate = UserType('Crate', Surface) # Caja (Tipo de superficie)

```

```

for tipo_de_objeto in [Place, Locatable, Depot, Distributor, Truck, Hoist,
↳Surface, Pallet, Crate]:
    dominio_depot.user_types.append(tipo_de_objeto)

```

[20]: *# Predicados booleanos*

```

# El predicado AT representa que el ubicable x está en el lugar y
at = Fluent('AT', BoolType(), x=Locatable, y=Place)
# El predicado ON representa que la caja x está sobre la superficie y
on = Fluent('ON', BoolType(), x=Crate, y=Surface)
# El predicado IN representa que la caja x está en el camión y
# (Nótese el guión bajo incluido en el nombre de la variable, ya que no se
# puede usar in, al tratarse de un identificador reservado de Python)
in_ = Fluent('IN', BoolType(), x=Crate, y=Truck)
# El predicado LIFTING representa que el polipasto x está levantando la caja y
lifting = Fluent('LIFTING', BoolType(), x=Hoist, y=Crate)
# El predicado AVAILABLE representa que el polipasto x está disponible
available = Fluent('AVAILABLE', BoolType(), x=Hoist)
# El predicado CLEAR representa que la superficie x está despejada
clear = Fluent('CLEAR', BoolType(), x=Surface)

for fluente in [at, on, in_, lifting, available, clear]:
    dominio_depot.add_fluent(fluente, default_initial_value=False)

```

[23]: *# Esquemas de acciones*

```

# La acción DRIVE representa que el camión x va del lugar y al lugar z
drive = InstantaneousAction('DRIVE', x=Truck, y=Place, z=Place)
x = drive.x
y = drive.y
z = drive.z
for hecho in [at(x,y)]:
    drive.add_precondition(hecho)
for hecho in [at(x,y)]:
    drive.add_effect(hecho, False)
for hecho in [at(x,z)]:
    drive.add_effect(hecho, True)

# La acción LIFT representa que el polipasto x levanta la caja y que se
# encontraba sobre la superficie z en el lugar p
lift = InstantaneousAction('LIFT', x=Hoist, y=Crate, z=Surface, p=Place)
x = lift.x
y = lift.y
z = lift.z
p = lift.p
for hecho in [at(x,p), at(z,p), on(y,z), available(x), clear(y)]:
    lift.add_precondition(hecho)

```

```

for hecho in [available(x), on(y,z), clear(y)]:
    lift.add_effect(hecho, False)
for hecho in [lifting(x,y), clear(z)]:
    lift.add_effect(hecho, True)

# La acción DROP representa que el polipasto x deja la caja y sobre la
# superficie z en el lugar p
drop = InstantaneousAction('DROP', x=Hoist, y=Crate, z=Surface, p=Place)
x = drop.x
y = drop.y
z = drop.z
p = drop.p
for hecho in [at(x,p), at(z,p), lifting(x,y), clear(z)]:
    drop.add_precondition(hecho)
for hecho in [clear(z), lifting(x,y)]:
    drop.add_effect(hecho, False)
for hecho in [clear(y), available(x), on(y,z), at(y,p)]:
    drop.add_effect(hecho, True)

# La acción LOAD representa que el polipasto x carga la caja y en el
# camión z en el lugar p
load = InstantaneousAction('LOAD', x=Hoist, y=Crate, z=Truck, p=Place)
x = load.x
y = load.y
z = load.z
p = load.p
for hecho in [at(x,p), at(y,p), at(z,p), lifting(x,y)]:
    load.add_precondition(hecho)
for hecho in [lifting(x,y)]:
    load.add_effect(hecho, False)
for hecho in [available(x), in_(y,z)]:
    load.add_effect(hecho, True)

# La acción UNLOAD representa que el polipasto x descarga la caja y del
# camión z en el lugar p
unload = InstantaneousAction('UNLOAD', x=Hoist, y=Crate, z=Truck, p=Place)
x = unload.x
y = unload.y
z = unload.z
p = unload.p
for hecho in [at(x,p), at(z,p), in_(y,z), available(x)]:
    unload.add_precondition(hecho)
for hecho in [in_(y,z), available(x)]:
    unload.add_effect(hecho, False)
for hecho in [lifting(x,y)]:
    unload.add_effect(hecho, True)

```

```
# Limpiamos ejecuciones anteriores
dominio_depot.clear_actions()
dominio_depot.add_actions([drive, lift, drop, load, unload])
```

```
[25]: import time
from unified_planning.io import PDDLReader
from unified_planning.shortcuts import OneshotPlanner

DOMINIO_DEPOT = "Depot/dominio_problem.pddl"
TIEMPO_MAXIMO = 30
ESTADOS_RESUELTOS = {"SOLVED_SATISFICING", "SOLVED_OPTIMALLY"}

def nombre_estado(resultado):
    estado = resultado.status
    return getattr(estado, "name", str(estado))

def resolver_instancia(lector, archivo):
    problema = lector.parse_problem(DOMINIO_DEPOT, archivo)
    with OneshotPlanner(name="fast-downward") as planner:
        inicio = time.time()
        resultado = planner.solve(problema, timeout=TIEMPO_MAXIMO)
        tiempo_total = time.time() - inicio

    estado = nombre_estado(resultado)
    print(f"Resultado para {archivo}: {estado}")
    print(f"Tiempo empleado: {tiempo_total:.2f} s")

    if estado in ESTADOS_RESUELTOS and resultado.plan is not None:
        print(f"Longitud del plan: {len(resultado.plan.actions)}")
    else:
        print("Sin solución en el tiempo límite o estado no resuelto.")

lector_pddl = PDDLReader()
lista_archivos = [f"Depot/pfile{i}" for i in range(1, 23)]

for archivo in lista_archivos:
    resolver_instancia(lector_pddl, archivo)
```

```
Resultado para Depot/pfile1: SOLVED_SATISFICING
Tiempo empleado: 0.40 s
Longitud del plan: 4
Resultado para Depot/pfile2: SOLVED_SATISFICING
Tiempo empleado: 0.47 s
Longitud del plan: 6
Resultado para Depot/pfile3: SOLVED_SATISFICING
Tiempo empleado: 0.51 s
Longitud del plan: 12
```

Resultado para Depot/pfile4: SOLVED_SATISFICING
Tiempo empleado: 0.70 s
Longitud del plan: 12
Resultado para Depot/pfile5: SOLVED_SATISFICING
Tiempo empleado: 0.67 s
Longitud del plan: 18
Resultado para Depot/pfile6: SOLVED_SATISFICING
Tiempo empleado: 0.98 s
Longitud del plan: 22
Resultado para Depot/pfile7: SOLVED_SATISFICING
Tiempo empleado: 0.51 s
Longitud del plan: 10
Resultado para Depot/pfile8: SOLVED_SATISFICING
Tiempo empleado: 0.68 s
Longitud del plan: 14
Resultado para Depot/pfile9: SOLVED_SATISFICING
Tiempo empleado: 1.06 s
Longitud del plan: 26
Resultado para Depot/pfile10: SOLVED_SATISFICING
Tiempo empleado: 0.77 s
Longitud del plan: 8
Resultado para Depot/pfile11: SOLVED_SATISFICING
Tiempo empleado: 1.33 s
Longitud del plan: 20
Resultado para Depot/pfile12: SOLVED_SATISFICING
Tiempo empleado: 2.61 s
Longitud del plan: 22
Resultado para Depot/pfile13: SOLVED_SATISFICING
Tiempo empleado: 0.79 s
Longitud del plan: 12
Resultado para Depot/pfile14: SOLVED_SATISFICING
Tiempo empleado: 1.38 s
Longitud del plan: 8
Resultado para Depot/pfile15: SOLVED_SATISFICING
Tiempo empleado: 2.82 s
Longitud del plan: 28
Resultado para Depot/pfile16: SOLVED_SATISFICING
Tiempo empleado: 0.96 s
Longitud del plan: 10
Resultado para Depot/pfile17: SOLVED_SATISFICING
Tiempo empleado: 1.64 s
Longitud del plan: 12
Resultado para Depot/pfile18: SOLVED_SATISFICING
Tiempo empleado: 2.81 s
Longitud del plan: 26
Resultado para Depot/pfile19: SOLVED_SATISFICING
Tiempo empleado: 1.59 s
Longitud del plan: 14

Resultado para Depot/pfile20: SOLVED_SATISFICING
Tiempo empleado: 4.68 s
Longitud del plan: 26
Resultado para Depot/pfile21: SOLVED_SATISFICING
Tiempo empleado: 6.30 s
Longitud del plan: 14
Resultado para Depot/pfile22: SOLVED_SATISFICING
Tiempo empleado: 21.90 s
Longitud del plan: 34

4.0.3 Ejercicio 3

[Sokoban](#) es un videojuego clásico de tipo puzle. En este juego el objetivo es empujar cajas, u otro tipo de objetos, en un almacén hasta llevarlos a las ubicaciones de almacenamiento. El juego se ve desde una perspectiva cenital. Los objetos solo se pueden empujar, nunca tirar de ellos, y solo un objeto se puede empujar a la vez. El desafío principal es planificar movimientos correctamente para evitar causar un punto muerto, una situación en la que un objeto o el jugador queda atrapado permanentemente, haciendo que el rompecabezas sea irresoluble.

En este ejercicio se pide lo siguiente:

1. Construir un dominio de planificación automática para el juego del Sokoban. Ese dominio debe contener los siguientes elementos:
 - Tipos de objetos: `thing`, `location`, `direction`, `player` (subtipo de `thing`), `stone` (subtipo de `thing`).
 - Predicados:
 - `CLEAR`: representa que una determinada localización (`location`) no contiene ninguna cosa (`thing`).
 - `AT`: representa que una cosa (`thing`) está en una determinada localización (`location`).
 - `AT-GOAL`: representa que una piedra (`stone`) está en una localización objetivo.
 - `IS-GOAL`: representa que una determinada localización (`location`) es una localización objetivo.
 - `IS-NONGOAL`: representa que una determinada localización (`location`) no es una localización objetivo.
 - `MOVE-DIR`: representa que se puede pasar de una determinada localización (`location`) a otra localización (`location`) adyacente moviéndose en una cierta dirección (`direction`).
 - Acciones:
 - `MOVE`: representa que el jugador (`player`) se mueve de la localización (`location`) que ocupa a una localización (`location`) libre adyacente en una determinada dirección (`direction`).
 - `PUSH-TO-NONGOAL`: representa que el jugador (`player`), estando en una determinada localización (`location`), empuja una piedra (`stone`) desde una localización (`location`) a otra localización (`location`) libre adyacente, que no es una localización objetivo, en una determinada dirección (`direction`).
 - `PUSH-TO-GOAL`: representa que el jugador (`player`), estando en una determinada localización (`location`), empuja una piedra (`stone`) desde una localización (`location`) a otra localización (`location`) libre adyacente, que es una localización

objetivo, en una determinada dirección (direction).

```
[26]: from unified_planning.shortcuts import *
from unified_planning.io import PDDLWriter, PDDLReader

# =====
# PARTE 1: CONSTRUIR EL DOMINIO (Cumpliendo el apartado 1 del Ejercicio)
# =====
dominio_sokoban = Problem("sokoban")

# 1. Tipos de objetos
Thing = UserType("thing")
Location = UserType("location")
Direction = UserType("direction")
Player = UserType("player", father=Thing)
Stone = UserType("stone", father=Thing)

# 2. Fluentes (predicados)
clear = Fluent("clear", BoolType(), l=Location)
at = Fluent("at", BoolType(), t=Thing, l=Location)
at_goal = Fluent("at-goal", BoolType(), s=Stone)
is_goal = Fluent("is-goal", BoolType(), l=Location)
is_nongoal = Fluent("is-nongoal", BoolType(), l=Location)
move_dir = Fluent("move-dir", BoolType(), l1=Location, l2=Location, d=Direction)

for f in [clear, at, at_goal, is_goal, is_nongoal, move_dir]:
    dominio_sokoban.add_fluent(f, default_initial_value=False)

# 3. Acción: MOVE
move = InstantaneousAction("move", p=Player, l1=Location, l2=Location,
    ↪d=Direction)
p, l1, l2, d = move.parameters
move.add_precondition(at(p, l1))
move.add_precondition(move_dir(l1, l2, d))
move.add_precondition(clear(l2))
move.add_effect(at(p, l1), False)
move.add_effect(at(p, l2), True)
move.add_effect(clear(l1), True)
move.add_effect(clear(l2), False)

# 4. Acción: PUSH-TO-NONGOAL
push_nogoal = InstantaneousAction(
    "push-to-nongoal",
    p=Player, l1=Location, s=Stone, l2=Location, l3=Location, d=Direction
)
p, l1, s, l2, l3, d = push_nogoal.parameters
push_nogoal.add_precondition(at(p, l1))
```

```

push_nogoal.add_precondition(at(s, 12))
push_nogoal.add_precondition(move_dir(l1, 12, d))
push_nogoal.add_precondition(move_dir(12, 13, d))
push_nogoal.add_precondition(clear(13))
push_nogoal.add_precondition(is_nongoal(13))
push_nogoal.add_effect(at(p, 11), False)
push_nogoal.add_effect(at(s, 12), False)
push_nogoal.add_effect(clear(12), False)
push_nogoal.add_effect(clear(13), False)
push_nogoal.add_effect(at_goal(s), False)
push_nogoal.add_effect(at(p, 12), True)
push_nogoal.add_effect(at(s, 13), True)
push_nogoal.add_effect(clear(11), True)

# 5. Acción: PUSH-TO-GOAL
push_goal = InstantaneousAction(
    "push-to-goal",
    p=Player, l1=Location, s=Stone, l2=Location, l3=Location, d=Direction
)
p, l1, s, l2, l3, d = push_goal.parameters
push_goal.add_precondition(at(p, 11))
push_goal.add_precondition(at(s, 12))
push_goal.add_precondition(move_dir(l1, 12, d))
push_goal.add_precondition(move_dir(12, 13, d))
push_goal.add_precondition(clear(13))
push_goal.add_precondition(is_goal(13))
push_goal.add_effect(at(p, 11), False)
push_goal.add_effect(at(s, 12), False)
push_goal.add_effect(clear(12), False)
push_goal.add_effect(clear(13), False)
push_goal.add_effect(at(p, 12), True)
push_goal.add_effect(at(s, 13), True)
push_goal.add_effect(clear(11), True)
push_goal.add_effect(at_goal(s), True)

dominio_sokoban.add_actions([move, push_nogoal, push_goal])

# Exportamos el dominio a PDDL
PDDLWriter(dominio_sokoban).write_domain("Sokoban/dominio_limpio.pddl")

```

2. Usar el algoritmo A* y la heurística $h^{\max}$ para resolver, con la menor cantidad posible de movimientos de empuje, los puzles que se encuentran en la carpeta Sokoban. Para ello, asignar coste 1 a las acciones PUSH-TO-NONGOAL y PUSH-TO-GOAL y coste 0 al resto de acciones.

[27]:

```

# =====
# PARTE 2: LEER, CONFIGURAR Y RESOLVER
# =====

```

```

lector = PDDLReader()
problema_instancia = lector.parse_problem("Sokoban/dominio_limpio.pddl", ↵
↵ "Sokoban/p01.pddl")

# Coste 1 para acciones de empuje y 0 para el resto
metrica = MinimizeActionCosts(
    {
        push_nogoal: Int(1),
        push_goal: Int(1),
    },
    default=Int(0),
)
problema_instancia.add_quality_metric(metrica)

estados_resueltos = {"SOLVED_SATISFICING", "SOLVED_OPTIMALLY"}

with OneshotPlanner(name="fast-downward") as planner:
    if not planner.supports(problema_instancia.kind):
        print("El planificador no soporta este tipo de problema.")
    else:
        print("Pensando...")
        resultado = planner.solve(problema_instancia, timeout=60)
        estado = getattr(resultado.status, "name", str(resultado.status))

        if estado in estados_resueltos and resultado.plan is not None:
            num_empujes = sum(
                1 for paso in resultado.plan.actions
                if paso.action.name in {"push-to-nongoal", "push-to-goal"}
            )

            print("\n ¡Puzzle resuelto!")
            print(f" Movimientos de empuje (coste): {num_empujes}")
            print(f" Pasos totales del jugador: {len(resultado.plan.actions)}")
            print("\n--- SECUENCIA ---")
            for i, paso in enumerate(resultado.plan.actions, start=1):
                print(f"{i}. {paso}")
        else:
            print(f" No resuelto (Estado: {estado})")

```

Pensando...

```

¡Puzzle resuelto!
Movimientos de empuje (coste): 13
Pasos totales del jugador: 56

--- SECUENCIA ---
1. move(player-01, pos-5-5, pos-5-4, dir-up)

```

2. move(player-01, pos-5-4, pos-5-3, dir-up)
3. move(player-01, pos-5-3, pos-4-3, dir-left)
4. move(player-01, pos-4-3, pos-4-2, dir-up)
5. move(player-01, pos-4-2, pos-3-2, dir-left)
6. move(player-01, pos-3-2, pos-2-2, dir-left)
7. move(player-01, pos-2-2, pos-2-3, dir-down)
8. push-to-nongoal(player-01, pos-2-3, stone-01, pos-3-3, pos-4-3, dir-right)
9. move(player-01, pos-3-3, pos-3-4, dir-down)
10. push-to-nongoal(player-01, pos-3-4, stone-02, pos-4-4, pos-5-4, dir-right)
11. move(player-01, pos-4-4, pos-3-4, dir-left)
12. move(player-01, pos-3-4, pos-3-3, dir-up)
13. move(player-01, pos-3-3, pos-3-2, dir-up)
14. move(player-01, pos-3-2, pos-4-2, dir-right)
15. push-to-nongoal(player-01, pos-4-2, stone-01, pos-4-3, pos-4-4, dir-down)
16. move(player-01, pos-4-3, pos-5-3, dir-right)
17. push-to-nongoal(player-01, pos-5-3, stone-02, pos-5-4, pos-5-5, dir-down)
18. push-to-nongoal(player-01, pos-5-4, stone-02, pos-5-5, pos-5-6, dir-down)
19. move(player-01, pos-5-5, pos-6-5, dir-right)
20. move(player-01, pos-6-5, pos-6-6, dir-down)
21. move(player-01, pos-6-6, pos-6-7, dir-down)
22. move(player-01, pos-6-7, pos-5-7, dir-left)
23. move(player-01, pos-5-7, pos-4-7, dir-left)
24. move(player-01, pos-4-7, pos-4-6, dir-up)
25. push-to-goal(player-01, pos-4-6, stone-02, pos-5-6, pos-6-6, dir-right)
26. move(player-01, pos-5-6, pos-4-6, dir-left)
27. move(player-01, pos-4-6, pos-4-7, dir-down)
28. move(player-01, pos-4-7, pos-5-7, dir-right)
29. move(player-01, pos-5-7, pos-6-7, dir-right)
30. push-to-goal(player-01, pos-6-7, stone-02, pos-6-6, pos-6-5, dir-up)
31. move(player-01, pos-6-6, pos-5-6, dir-left)
32. move(player-01, pos-5-6, pos-5-5, dir-up)
33. move(player-01, pos-5-5, pos-5-4, dir-up)
34. move(player-01, pos-5-4, pos-5-3, dir-up)
35. move(player-01, pos-5-3, pos-4-3, dir-left)
36. move(player-01, pos-4-3, pos-3-3, dir-left)
37. move(player-01, pos-3-3, pos-3-4, dir-down)
38. push-to-nongoal(player-01, pos-3-4, stone-01, pos-4-4, pos-5-4, dir-right)
39. move(player-01, pos-4-4, pos-4-3, dir-up)
40. move(player-01, pos-4-3, pos-5-3, dir-right)
41. push-to-nongoal(player-01, pos-5-3, stone-01, pos-5-4, pos-5-5, dir-down)
42. push-to-nongoal(player-01, pos-5-4, stone-01, pos-5-5, pos-5-6, dir-down)
43. push-to-nongoal(player-01, pos-5-5, stone-02, pos-6-5, pos-7-5, dir-right)
44. move(player-01, pos-6-5, pos-6-6, dir-down)
45. move(player-01, pos-6-6, pos-6-7, dir-down)
46. move(player-01, pos-6-7, pos-5-7, dir-left)
47. move(player-01, pos-5-7, pos-4-7, dir-left)
48. move(player-01, pos-4-7, pos-4-6, dir-up)
49. push-to-goal(player-01, pos-4-6, stone-01, pos-5-6, pos-6-6, dir-right)

```

50. move(player-01, pos-5-6, pos-5-7, dir-down)
51. move(player-01, pos-5-7, pos-6-7, dir-right)
52. move(player-01, pos-6-7, pos-7-7, dir-right)
53. move(player-01, pos-7-7, pos-8-7, dir-right)
54. move(player-01, pos-8-7, pos-8-6, dir-up)
55. move(player-01, pos-8-6, pos-8-5, dir-up)
56. push-to-goal(player-01, pos-8-5, stone-02, pos-7-5, pos-6-5, dir-left)

```

4.0.4 Ejercicio 3

[Sokoban](#) es un videojuego clásico de tipo puzle. En este juego el objetivo es empujar cajas, u otro tipo de objetos, en un almacén hasta llevarlos a las ubicaciones de almacenamiento. El juego se ve desde una perspectiva cenital. Los objetos solo se pueden empujar, nunca tirar de ellos, y solo un objeto se puede empujar a la vez. El desafío principal es planificar movimientos correctamente para evitar causar un punto muerto, una situación en la que un objeto o el jugador queda atrapado permanentemente, haciendo que el rompecabezas sea irresoluble.

En este ejercicio se pide lo siguiente:

1. Construir un dominio de planificación automática para el juego del Sokoban. Ese dominio debe contener los siguientes elementos:
 - Tipos de objetos: `thing`, `location`, `direction`, `player` (subtipo de `thing`), `stone` (subtipo de `thing`).
 - Predicados:
 - `CLEAR`: representa que una determinada localización (`location`) no contiene ninguna cosa (`thing`).
 - `AT`: representa que una cosa (`thing`) está en una determinada localización (`location`).
 - `AT-GOAL`: representa que una piedra (`stone`) está en una localización objetivo.
 - `IS-GOAL`: representa que una determinada localización (`location`) es una localización objetivo.
 - `IS-NONGOAL`: representa que una determinada localización (`location`) no es una localización objetivo.
 - `MOVE-DIR`: representa que se puede pasar de una determinada localización (`location`) a otra localización (`location`) adyacente moviéndose en una cierta dirección (`direction`).
 - Acciones:
 - `MOVE`: representa que el jugador (`player`) se mueve de la localización (`location`) que ocupa a una localización (`location`) libre adyacente en una determinada dirección (`direction`).
 - `PUSH-TO-NONGOAL`: representa que el jugador (`player`), estando en una determinada localización (`location`), empuja una piedra (`stone`) desde una localización (`location`) a otra localización (`location`) libre adyacente, que no es una localización objetivo, en una determinada dirección (`direction`).
 - `PUSH-TO-GOAL`: representa que el jugador (`player`), estando en una determinada localización (`location`), empuja una piedra (`stone`) desde una localización (`location`) a otra localización (`location`) libre adyacente, que es una localización objetivo, en una determinada dirección (`direction`).

```

[32]: from unified_planning.shortcuts import *
      from unified_planning.io import PDDLWriter

      # =====
      # INICIO CODIGO ANADIDO: construccion del dominio Sokoban
      # Funcion en el hilo: define tipos, fluentes y acciones; exporta dominio PDDL
      # =====

      dominio_sokoban = Problem("sokoban")

      # 1) Tipos del enunciado
      thing = UserType("thing")
      location = UserType("location")
      direction = UserType("direction")
      player = UserType("player", father=thing)
      stone = UserType("stone", father=thing)

      # 2) Predicados del enunciado
      clear = Fluent("CLEAR", BoolType(), l=location)
      at = Fluent("AT", BoolType(), t=thing, l=location)
      at_goal = Fluent("AT-GOAL", BoolType(), s=stone)
      is_goal = Fluent("IS-GOAL", BoolType(), l=location)
      is_nongoal = Fluent("IS-NONGOAL", BoolType(), l=location)
      move_dir = Fluent("MOVE-DIR", BoolType(), l1=location, l2=location, d=direction)

      # total-cost aparece en los ficheros de instancia en el estado inicial
      total_cost = Fluent("total-cost", IntType())

      for fluente in [clear, at, at_goal, is_goal, is_nongoal, move_dir]:
          dominio_sokoban.add_fluent(fluente, default_initial_value=False)
      dominio_sokoban.add_fluent(total_cost, default_initial_value=Int(0))

      # 3) Accion MOVE (misma estructura simple que usan las instancias)
      move = InstantaneousAction("MOVE", p=player, l1=location, l2=location,
                                d=direction)
      p, l1, l2, d = move.parameters
      move.add_precondition(at(p, l1))
      move.add_precondition(move_dir(l1, l2, d))
      move.add_effect(at(p, l1), False)
      move.add_effect(at(p, l2), True)

      # 4) Accion PUSH-TO-NONGOAL
      push_to_nongoal = InstantaneousAction(
          "PUSH-TO-NONGOAL",
          p=player, l1=location, s=stone, l2=location, d=direction
      )
      p, l1, s, l2, d = push_to_nongoal.parameters

```

```

push_to_nongoal.add_precondition(at(p, l1))
push_to_nongoal.add_precondition(at(s, l1))
push_to_nongoal.add_precondition(move_dir(l1, l2, d))
push_to_nongoal.add_precondition(clear(l2))
push_to_nongoal.add_precondition(is_nongoal(l2))
push_to_nongoal.add_precondition(is_nongoal(l1))
push_to_nongoal.add_effect(at(p, l1), False)
push_to_nongoal.add_effect(at(s, l1), False)
push_to_nongoal.add_effect(clear(l2), False)
push_to_nongoal.add_effect(at_goal(s), False)
push_to_nongoal.add_effect(at(p, l2), True)
push_to_nongoal.add_effect(at(s, l2), True)

# 5) Accion PUSH-TO-GOAL
push_to_goal = InstantaneousAction(
    "PUSH-TO-GOAL",
    p=player, l1=location, s=stone, l2=location, d=direction
)
p, l1, s, l2, d = push_to_goal.parameters
push_to_goal.add_precondition(at(p, l1))
push_to_goal.add_precondition(at(s, l1))
push_to_goal.add_precondition(clear(l2))
push_to_goal.add_precondition(move_dir(l1, l2, d))
push_to_goal.add_precondition(is_nongoal(l1))
push_to_goal.add_precondition(is_goal(l2))
push_to_goal.add_effect(at(p, l1), False)
push_to_goal.add_effect(at(s, l1), False)
push_to_goal.add_effect(clear(l2), False)
push_to_goal.add_effect(at(p, l2), True)
push_to_goal.add_effect(at(s, l2), True)
push_to_goal.add_effect(at_goal(s), True)

dominio_sokoban.add_actions([move, push_to_nongoal, push_to_goal])

# Exportar dominio construido para usarlo en las instancias
PDDLWriter(dominio_sokoban).write_domain("Sokoban/dominio_limpio.pddl")
print("Dominio Sokoban exportado en Sokoban/dominio_limpio.pddl")

# =====
# FIN CODIGO ANADIDO
# =====

```

Dominio Sokoban exportado en Sokoban/dominio_limpio.pddl

2. Usar el algoritmo A* y la heurística $h^{\max}$ para resolver, con la menor cantidad posible de movimientos de empuje, los puzles que se encuentran en la carpeta Sokoban. Para ello, asignar coste 1 a las acciones PUSH-TO-NONGOAL y PUSH-TO-GOAL y coste 0 al resto de acciones.

```

[33]: import numpy as np
from unified_planning.io import PDDLReader
from unified_planning.shortcuts import OneshotPlanner, MinimizeActionCosts, Int

# =====
# INICIO CODIGO ANADIDO: resolucion de instancias Sokoban
# Funcion en el hilo: lee instancias, aplica metrica de empujes y planifica
# =====

lector = PDDLReader()
parametros_fd = {"fast_downward_search_config": "astar(hmax())"}
estados_resueltos = {"SOLVED_SATISFICING", "SOLVED_OPTIMALLY"}

resueltas = 0
intentadas = 0
errores_parseo = 0

def buscar_accion_por_nombre(problema, nombre_objetivo):
    nombre_objetivo = nombre_objetivo.lower()
    for accion in problema.actions:
        if accion.name.lower() == nombre_objetivo:
            return accion
    return None

# Recorreremos p01..p30
for i in range(1, 31):
    archivo_instancia = f"Sokoban/p{i:02d}.pddl"
    print(f"\n--- Intentando {archivo_instancia} ---")

    try:
        problema = lector.parse_problem("Sokoban/dominio_limpio.pddl",
↪archivo_instancia)
    except Exception as e:
        errores_parseo += 1
        print(f"Error de parseo: {e}")
        continue

    intentadas += 1

    # Coste 1 para empujes; coste 0 para el resto de acciones
    accion_push_nongoal = buscar_accion_por_nombre(problema, "push-to-nongoal")
    accion_push_goal = buscar_accion_por_nombre(problema, "push-to-goal")

    if accion_push_nongoal is None or accion_push_goal is None:
        print("No se encontraron acciones de empuje en la instancia.")
        continue

```

```

metrica = MinimizeActionCosts(
    {
        accion_push_nongoal: Int(1),
        accion_push_goal: Int(1),
    },
    default=Int(0),
)
problema.add_quality_metric(metrica)

with OneshotPlanner(name="fast-downward", params=parametros_fd) as planner:
    if not planner.supports(problema.kind):
        print("Planificador no compatible con el problema.")
        continue

    resultado = planner.solve(problema, timeout=60)
    estado = getattr(resultado.status, "name", str(resultado.status))

    if estado in estados_resueltos and resultado.plan is not None:
        nombres_acciones = np.array([paso.action.name.lower() for paso in
↳ resultado.plan.actions])
        num_empujes = int(np.sum(
            (nombres_acciones == "push-to-nongoal") |
            (nombres_acciones == "push-to-goal")
        ))

        print(f"Estado: {estado}")
        print(f"Empujes (coste): {num_empujes}")
        print(f"Pasos totales: {len(resultado.plan.actions)}")
        resueltas += 1
    else:
        print(f"No resuelto. Estado: {estado}")

print("\n===== RESUMEN =====")
print(f"Instancias parseadas/intentas: {intentadas}")
print(f"Instancias resueltas: {resueltas}")
print(f"Errores de parseo: {errores_parseo}")
print("=====")

# =====
# FIN CODIGO ANADIDO
# =====

```

```

--- Intentando Sokoban/p01.pddl ---
Planificador no compatible con el problema.

```

```

--- Intentando Sokoban/p02.pddl ---
No resuelto. Estado: UNSOLVABLE_INCOMPLETELY

```

--- Intentando Sokoban/p03.pddl ---
Estado: SOLVED_SATISFICING
Empujes (coste): 2
Pasos totales: 4

--- Intentando Sokoban/p04.pddl ---
No resuelto. Estado: UNSOLVABLE_INCOMPLETELY

--- Intentando Sokoban/p05.pddl ---
Estado: SOLVED_SATISFICING
Empujes (coste): 8
Pasos totales: 21

--- Intentando Sokoban/p06.pddl ---
Estado: SOLVED_SATISFICING
Empujes (coste): 5
Pasos totales: 6

--- Intentando Sokoban/p07.pddl ---
Estado: SOLVED_SATISFICING
Empujes (coste): 3
Pasos totales: 14

--- Intentando Sokoban/p08.pddl ---
No resuelto. Estado: UNSOLVABLE_INCOMPLETELY

--- Intentando Sokoban/p09.pddl ---
No resuelto. Estado: UNSOLVABLE_INCOMPLETELY

--- Intentando Sokoban/p10.pddl ---
No resuelto. Estado: UNSOLVABLE_INCOMPLETELY

--- Intentando Sokoban/p11.pddl ---
Estado: SOLVED_SATISFICING
Empujes (coste): 25
Pasos totales: 171

--- Intentando Sokoban/p12.pddl ---
No resuelto. Estado: UNSOLVABLE_INCOMPLETELY

--- Intentando Sokoban/p13.pddl ---
Estado: SOLVED_SATISFICING
Empujes (coste): 4
Pasos totales: 9

--- Intentando Sokoban/p14.pddl ---
No resuelto. Estado: UNSOLVABLE_INCOMPLETELY

--- Intentando Sokoban/p15.pddl ---
Estado: SOLVED_SATISFICING
Empujes (coste): 16
Pasos totales: 59

--- Intentando Sokoban/p16.pddl ---
No resuelto. Estado: UNSOLVABLE_INCOMPLETELY

--- Intentando Sokoban/p17.pddl ---
No resuelto. Estado: UNSOLVABLE_INCOMPLETELY

--- Intentando Sokoban/p18.pddl ---
No resuelto. Estado: UNSOLVABLE_INCOMPLETELY

--- Intentando Sokoban/p19.pddl ---
No resuelto. Estado: UNSOLVABLE_INCOMPLETELY

--- Intentando Sokoban/p20.pddl ---
Estado: SOLVED_SATISFICING
Empujes (coste): 2
Pasos totales: 4

--- Intentando Sokoban/p21.pddl ---
Estado: SOLVED_SATISFICING
Empujes (coste): 10
Pasos totales: 18

--- Intentando Sokoban/p22.pddl ---
No resuelto. Estado: UNSOLVABLE_INCOMPLETELY

--- Intentando Sokoban/p23.pddl ---
Estado: SOLVED_SATISFICING
Empujes (coste): 7
Pasos totales: 22

--- Intentando Sokoban/p24.pddl ---
No resuelto. Estado: UNSOLVABLE_INCOMPLETELY

--- Intentando Sokoban/p25.pddl ---
No resuelto. Estado: UNSOLVABLE_INCOMPLETELY

--- Intentando Sokoban/p26.pddl ---
No resuelto. Estado: UNSOLVABLE_INCOMPLETELY

--- Intentando Sokoban/p27.pddl ---
Estado: SOLVED_SATISFICING
Empujes (coste): 7

Pasos totales: 20

--- Intentando Sokoban/p28.pddl ---

No resuelto. Estado: UNSOLVABLE_INCOMPLETELY

--- Intentando Sokoban/p29.pddl ---

Estado: SOLVED_SATISFICING

Empujes (coste): 12

Pasos totales: 32

--- Intentando Sokoban/p30.pddl ---

No resuelto. Estado: TIMEOUT

===== RESUMEN =====

Instancias parseadas/intentas: 30

Instancias resueltas: 12

Errores de parseo: 0

=====